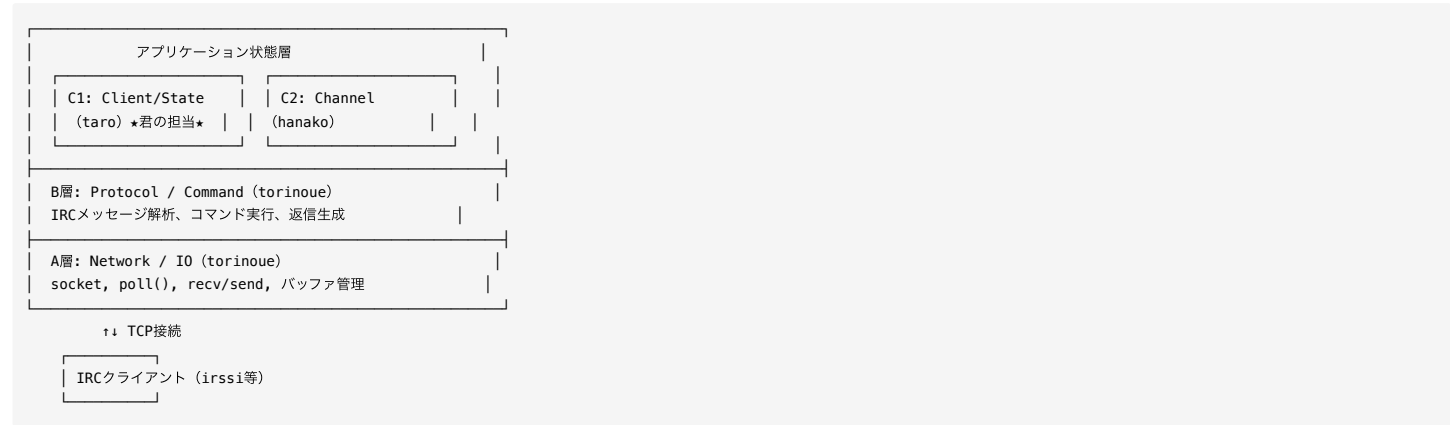


オンボーディング: C1担当（taro）

作成日: 2026-05-23
対象: taro（C1担当: Client / ServerState）
所要時間: 約30分で読める

1. 全体アーキテクチャ（4層）

OSI参照モデル準拠: アプリケーション層（抽象度高）が上、ネットワーク層（低レイヤー）が下。



データの流れ（上向き: 受信 / 下向き: 送信）:

【受信】 クライアント → A層(recv) → B層(parse/dispatch) → C1/C2層(状態更新)
【送信】 C1/C2層 → B層(reply生成) → A層(send) → クライアント

2. 君の担当: C1層

担当クラス

クラス	役割
Client	1人のIRCユーザーの状態（nick, username, 認証状態など）
ServerState	サーバー全体の辞書管理（fd→Client, nick→Client, channel→Channel）

Clientが持つもの

```
class Client {
    int _fd; // 接続のファイルディスクリプタ
    std::string _nick; // ニックネーム (例: "taro")
    std::string _username; // ユーザー名
    std::string _realname; // 本名
    std::string _host; // 接続元ホスト (例: "127.0.0.1")
    bool _passOk; // PASSコマンド成功したか
    bool _registered; // 登録完了したか (PASS+NICK+USER全部済み)

public:
    // ゲッター (get prefix統一)
    int getFd() const;
    std::string getNick() const;
    std::string getUsername() const;
    std::string getRealname() const;
    std::string getHost() const;
    std::string getFullPrefix() const; // "nick!user@host" を返す
    bool isRegistered() const;
};
```

getFullPrefix() について

サーバーがクライアントにメッセージを送る際、送信元を `nick!user@host` 形式で付与する。

// 例: taro が #room で "hello" と発言
// サーバーは #room の他メンバーに以下を送信:
":taro!taro@127.0.0.1 PRIVMSG #room :hello\r\n"

RFC根拠: RFC 1459 Section 2.3, Note 6（詳細は `rfc1459_prefix_analysis.md` 参照）

ServerStateが持つもの

```
class ServerState {
    std::string                _password; // サーバパスワード
    std::map<int, Client*>      _fdToClient; // fd → Client
    std::map<std::string, Client*> _nickToClient; // nick → Client
    std::map<std::string, Channel*> _channels; // channel名 → Channel
};
```

重要ルール

【実装】nick変更は必ずServerState経由:

```
// NG: 辞書が壊れる
client.setNick("newNick");

// OK: 辞書も同時更新
state.updateNick(client, "newNick");
```

【設計】Client削除もServerState経由（IRCの一般的動作を反映）:

```
// ServerState::removeClient(fd) が以下を自動処理:
// - 全Channelからmember/operator/invited削除
// - fd辞書・nick辞書の更新
// - 空Channelの削除
```

3. IRC登録フロー（C1の主戦場）

クライアントがサーバーに接続後、以下の順でコマンドを送る:

1. PASS <password> → パスワード認証
2. NICK <nickname> → ニックネーム設定
3. USER <user> ... → ユーザー情報設定

3つ全部成功して初めて「登録完了」。それまでJOINやPRIVMSGは使えない。

```
状態遷移:
【未認証】 → PASS成功 → 【認証済】 → NICK+USER成功 → 【登録完了】
```

4. 読むべきドキュメント（この順番で）

順序	ファイル	内容	時間目安
1	このファイル	全体像把握	10分
2	reading_guide_common.md	共通概念、データフロー図	10分
3	design.md Section 3.3, 5, 6	Client/ServerStateの詳細設計	20分
4	interface.md Section 7	Client/ServerStateの関数一覧	15分
5	reading_guide_C1.md	C1向け詳細ガイド	10分

RFC（後で読む）:

- RFC 1459 Section 4.1（PASS/NICK/USER）
- RFC 2812 Section 3.1（登録コマンド詳細）

5. よくある疑問

Q: ConnectionとClientの違いは？

A:

- Connection（A層）: TCP接続そのもの。recv/sendバッファを持つ
- Client（C1層）: IRCユーザーの論理状態。nick/username等を持つ

両者はfdで紐づくが、責務は分離。

Q: なぜnick変更をServerState経由にする？

A: nick → Client の辞書を同時更新しないと、古いnickで検索したとき壊れるから。

Q: Channelとの関係は？

A: Client* はChannelのmember/operator/invitedリストで参照される。Client削除時は全Channelから参照を消す必要がある（ServerState::removeClient() が自動処理）。